

NP-Completeness: uma reorganização didática*

PCC104 – Projeto e Análise de Algoritmos

14 de julho de 2026

Sumário

1	Motivação: por que precisamos disso	1
2	Problemas de decisão e a classe P	2
3	A classe NP: fácil de <i>verificar</i>	2
4	Reduções em tempo polinomial	3
5	NP-completeness: definição	3
6	O Teorema de Cook-Levin (como caixa-preta)	4
7	A receita prática e a cadeia de reduções	4
7.1	3-CNF-SAT $\leq_p$ CLIQUE (completa)	5
7.2	CLIQUE $\leq_p$ VERTEX-COVER (completa)	5
7.3	As demais reduções da cadeia (esboço)	6
8	E agora? Vivendo com problemas NP-completos	6

1 Motivação: por que precisamos disso

Até aqui, vimos uma sequência de problemas que admitem algoritmos eficientes: ordenação ($O(n \log n)$), caminho mínimo (Dijkstra, $O(E + V \log V)$), fluxo máximo, árvore geradora mínima. Em todos esses casos, o tempo de execução cresce como um polinômio no tamanho da entrada, e essa é precisamente a noção que usamos para dizer que um problema é **tratável na prática**.

Mas existe uma classe de problemas, estruturalmente muito parecidos com os anteriores, para os quais *ninguém jamais encontrou* um algoritmo polinomial – apesar de décadas de tentativas por parte de gente muito competente. Compare:

Fácil (em P)	Aparentemente difícil
Existe um <i>caminho</i> de s a t com no máximo k arestas? (BFS, $O(V + E)$)	Existe um <i>caminho simples</i> (sem repetir vértices) de s a t que passe por <i>todos</i> os vértices? (HAM-PATH)
Existe um circuito de Euler (percorre toda <i>aresta</i> exatamente uma vez)? ($O(V + E)$)	Existe um ciclo hamiltoniano (visita toda <i>vértice</i> exatamente uma vez)? (HAM-CYCLE)
2-SAT: fórmula booleana em FNC ¹ com 2 literais por cláusula é satisfazível? ($O(n + m)$, via componentes fortemente conexas)	3-SAT: mesma pergunta, com 3 literais por cláusula

A diferença de enunciado é mínima. A diferença de dificuldade prática é gigantesca. Isso não é acidente – é o objeto de estudo desta aula.

Por que isso importa na prática? Problemas como escalonamento de tarefas, roteamento de veículos, alocação de recursos, design de circuitos e bioinformática recaem, quase todos, nessa categoria difícil. Um profissional que não reconhece um problema NP-completo corre o risco de gastar meses tentando achar um algoritmo polinomial que (com altíssima probabilidade) não existe, em vez de partir direto para heurísticas, aproximações ou formulações exatas com solvers especializados.

*Material baseado em (Cormen et al., 2009), com referência aos trabalhos originais de Cook (1971), Karp (1972) e Garey and Johnson (1979). A ordem de apresentação aqui difere deliberadamente da seção 34 de Cormen et al. (2009): o formalismo de codificações/problemas abstratos é reduzido ao mínimo necessário, a intuição de reduções é construída antes da definição formal, e a demonstração de Cook-Levin é tratada como resultado de “caixa-preta”, com foco no uso prático das reduções e no que fazer diante de um problema NP-completo.

¹FNC, literal e cláusula serão definidos formalmente na Definição 7.1; por ora, basta a ideia: uma fórmula que é um “E” de cláusulas, cada cláusula um “OU” de variáveis possivelmente negadas.

n	n^3 (polinomial)	2^n (exponencial)
10	1.000	1.024
20	8.000	1.048.576
30	27.000	$\approx 10^9$
50	125.000	$\approx 10^{15}$
60	216.000	$\approx 10^{18}$ (idade do universo em segundos: $\approx 4 \times 10^{17}$)

Tabela 1: Crescimento polinomial vs. exponencial. Para $n = 60$, um algoritmo exponencial já é impraticável mesmo em supercomputadores.

O objetivo desta aula não é provar que $P \neq NP$ – isso é um problema em aberto, um dos Problemas do Milênio do Clay Mathematics Institute. O objetivo é: (i) formalizar a diferença entre “fácil de resolver” e “fácil de verificar”; (ii) aprender a técnica de *reduções* para provar que um problema é “tão difícil quanto” outro; (iii) saber reconhecer um problema NP-completo quando você encontrar um; (iv) saber o que fazer depois de reconhecê-lo.

2 Problemas de decisão e a classe P

Para comparar problemas entre si de forma rigorosa, é conveniente reduzir todo problema a uma pergunta de **sim/não**.

Notação. Denotamos por $\{0, 1\}^*$ o conjunto de todas as **cadeias binárias finitas** (incluindo a cadeia vazia): $\{0, 1\}^* = \{\epsilon, 0, 1, 00, 01, 10, 11, 000, \dots\}$. Para uma cadeia $x \in \{0, 1\}^*$, $|x|$ denota seu comprimento em bits.

Definição 2.1 (Problema de decisão). Um problema de decisão é uma função $f : \{0, 1\}^* \rightarrow \{0, 1\}$. Formalmente, identificamos o problema com a **linguagem** $L = \{x \in \{0, 1\}^* : f(x) = 1\}$ – o conjunto das instâncias codificadas cuja resposta é “sim”. Dizemos que um algoritmo **decide** L se, para toda entrada x , ele responde corretamente se $x \in L$ ou $x \notin L$.

Exemplo 2.1. PATH: dado um grafo G , vértices u, v e inteiro k , existe caminho de u a v com no máximo k arestas? A versão de otimização (“qual o comprimento do caminho mínimo?”) é equivalente em dificuldade, no seguinte sentido: quem resolve a otimização resolve a decisão de graça (compare o mínimo com k); e quem resolve a decisão resolve a otimização com busca binária sobre k – um número apenas polinomial de chamadas. Por isso, restringir a teoria a problemas de decisão *não perde generalidade* para nossos fins.

Toda entrada – grafo, fórmula booleana, conjunto de números – é assumida codificada como uma cadeia de bits por um esquema de codificação razoável (sem *blow-up* exponencial: números em binário, não em unário). Não vamos nos aprofundar nesse ponto – ele é relevante apenas para evitar patologias como o algoritmo pseudo-polinomial de SUBSET-SUM, que reencontraremos na Seção 7.

Definição 2.2 (Classe P). P é o conjunto das linguagens L para as quais existe um algoritmo que decide L em tempo $O(n^k)$ para alguma constante k , onde $n = |x|$ é o tamanho da entrada.

P é o nosso modelo formal de “tratável”. Tudo que vimos no curso até agora está em P .

3 A classe NP: fácil de *verificar*

Considere HAM-CYCLE: dado um grafo G , existe um **ciclo hamiltoniano** – um ciclo simples que visita cada vértice de G exatamente uma vez? Ninguém conhece algoritmo polinomial para resolver isso. Mas note uma assimetria interessante: se alguém te der um *ciclo candidato*, verificar se ele é de fato hamiltoniano é trivialíssimo – percorra a sequência de vértices, confira que cada aresta consecutiva existe no grafo e que cada vértice aparece exatamente uma vez. Isso é $O(V + E)$.

Essa assimetria – difícil de *resolver*, fácil de *verificar dada uma resposta pronta* – é o conceito central desta seção.

Definição 3.1 (Verificador). Um algoritmo $A(x, y)$ de duas entradas é um **verificador** para a linguagem L se

$$L = \{x \in \{0, 1\}^* : \exists y \in \{0, 1\}^* \text{ com } |y| = O(|x|^c), A(x, y) \text{ aceita}\},$$

para alguma constante c . Chamamos y de **certificado** (ou testemunha). A é um **verificador de tempo polinomial** se roda em tempo polinomial em $|x|$. Note que exigimos o certificado curto ($|y|$ polinomial em $|x|$); sem essa exigência, o verificador nem conseguiria ler y inteiro em tempo polinomial em $|x|$.

Definição 3.2 (Classe NP). NP é o conjunto das linguagens que possuem um verificador de tempo polinomial.

Em palavras: $L \in NP$ se toda instância “sim” de L possui uma *prova curta e rapidamente checável* de que a resposta é sim. Nada é dito sobre instâncias “não” – para elas, pode não existir prova curta nenhuma.

Exemplo 3.1. Para HAM-CYCLE, o certificado y é a própria sequência de vértices do ciclo proposto. Para CLIQUE (G tem um subconjunto de k vértices mutuamente adjacentes?), o certificado é o próprio subconjunto de vértices – o verificador testa os $\binom{k}{2}$ pares. Para SAT (uma fórmula booleana tem atribuição de valores-verdade que a torna verdadeira?), o certificado é a atribuição – o verificador avalia a fórmula.

Observação 3.1. NP não significa “não-polinomial” – é um erro de leitura comum. $NP = Nondeterministic Polynomial time$: o nome vem de uma definição equivalente via máquinas de Turing não-determinísticas, que “adivinham” o certificado. Preferimos aqui a definição via verificador, mais intuitiva.

Teorema 3.1. $P \subseteq NP$.

Ideia. Se $L \in P$, construa um verificador que ignora y e decide x diretamente com o algoritmo polinomial de L . Certificado vazio, verificação trivialmente correta. $\square$

A grande pergunta em aberto: $P = NP$? Isto é, será que todo problema fácil de verificar é também fácil de resolver? A crença quase unânime da comunidade é que $P \neq NP$, mas ninguém sabe provar. O restante desta aula constrói a maquinaria para identificar os problemas *mais difíceis* de NP – se qualquer um deles cair em P , então $P = NP$.

4 Reduções em tempo polinomial

Antes de qualquer definição formal, um exemplo concreto. Considere dois problemas:

- $CLIQUE(G, k)$: G tem um subconjunto de k vértices todos mutuamente adjacentes?
- $INDEPENDENT-SET(G, k)$: G tem um subconjunto de k vértices em que *nenhum* par é adjacente?

Seja $\bar{G}$ o **grafo complementar** de G : mesmo conjunto de vértices, e u, v são adjacentes em $\bar{G}$ se, e somente se, *não* são adjacentes em G . É imediato que:

$$G \text{ tem clique de tamanho } k \iff \bar{G} \text{ tem conjunto independente de tamanho } k,$$

pois um conjunto de vértices é totalmente conectado em G exatamente quando é totalmente desconectado em $\bar{G}$.

Isso nos dá uma transformação: dado (G, k) , calcule $\bar{G}$ (tempo $O(V^2)$, certamente polinomial) e pergunte sobre $(\bar{G}, k)$.

A resposta é a mesma para os dois problemas. Chamamos isso de *redução*.

Definição 4.1 (Redução em tempo polinomial). Dizemos que $L_1 \leq_p L_2$ (L_1 **reduz em tempo polinomial** a L_2) se existe uma função $f : \{0, 1\}^* \rightarrow \{0, 1\}^*$, computável em tempo polinomial, tal que

$$x \in L_1 \iff f(x) \in L_2, \quad \text{para todo } x \in \{0, 1\}^*.$$

A leitura intuitiva do símbolo: $L_1 \leq_p L_2$ significa “ L_1 não é mais difícil que L_2 ” – quem sabe resolver L_2 automaticamente sabe resolver L_1 .

Teorema 4.1. Se $L_1 \leq_p L_2$ e $L_2 \in P$, então $L_1 \in P$.

Demonstração. Para decidir $x \in L_1$: calcule $f(x)$ (tempo polinomial) e decida se $f(x) \in L_2$ (tempo polinomial, pois $L_2 \in P$). Um detalhe que merece atenção: como f roda em tempo polinomial em $|x|$, sua saída satisfaz $|f(x)| = O(|x|^c)$ – um algoritmo não consegue escrever mais símbolos do que o tempo de que dispõe. Assim, o custo do segundo passo é polinomial em $|f(x)|$, que é polinomial em $|x|$; e a composição de polinômios é um polinômio. $\square$

Este é o único fato que você precisa para entender por que reduções importam. Leia o Teorema 4.1 na contrapositiva: se $L_1 \notin P$, então $L_2 \notin P$. Ou seja: *reduzir L_1 a L_2 transfere a dificuldade de L_1 para L_2 .*

Observação 4.1 (O erro mais comum de estudante). A direção da redução importa, e é contraintuitiva na primeira vez. Para provar que L_2 é *difícil*, você reduz um problema *já conhecido como difícil* para L_2 (ou seja, mostra $L_1 \leq_p L_2$ com L_1 sabidamente difícil). Você **não** reduz L_2 para outra coisa – isso só mostraria que L_2 é *fácil demais*, não que é difícil. A pergunta que a redução responde é: “se eu soubesse resolver L_2 rapidamente, eu resolveria L_1 rapidamente também?”. Se sim, L_2 é pelo menos tão difícil quanto L_1 .

Lema 4.2 (Transitividade). Se $L_1 \leq_p L_2$ e $L_2 \leq_p L_3$, então $L_1 \leq_p L_3$.

Ideia. Componha as duas funções de redução; pelo mesmo argumento do Teorema 4.1, a composição ainda é computável em tempo polinomial. $\square$

Essa transitividade é o que vai nos permitir construir *cadeias* de reduções na Seção 7, sem provar cada problema “do zero”.

5 NP-completude: definição

Agora podemos formalizar a noção de “problema mais difícil de NP ”.

Definição 5.1 (NP-completo e NP-difícil). Uma linguagem L é **NP-difícil** (*NP-hard*) se $L' \leq_p L$ para *toda* linguagem $L' \in NP$. Uma linguagem L é **NP-completa** se, além de NP-difícil, também vale $L \in NP$.

Note: NP-difícil *não* exige pertencer a NP – existem problemas NP-difíceis que (acredita-se) estão fora de NP , como a versão de otimização do TSP ou o problema da parada. Os NP-completos são exatamente os NP-difíceis que ainda vivem dentro de NP .

Teorema 5.1. Se algum problema NP-completo L está em P , então $P = NP$.

Ideia. Por definição de NP-completude, todo $L' \in NP$ satisfaz $L' \leq_p L$. Se $L \in P$, pelo Teorema 4.1, $L' \in P$ para todo $L' \in NP$. Logo $NP \subseteq P$; como já sabemos $P \subseteq NP$, segue $P = NP$. $\square$

Este teorema é a razão de todo o interesse prático da NP-completude: **os problemas NP-completos são, coletivamente, os “elos mais fracos” de NP** – resolver qualquer um em tempo polinomial resolveria todos. Como décadas de tentativas falharam para *milhares* de problemas NP-completos catalogados (Garey and Johnson, 1979), a crença de que $P \neq NP$ se fortalece a cada tentativa fracassada.

Corolário 5.2. Para mostrar que L é NP-completo, basta:

1. mostrar $L \in NP$ (exibir um verificador polinomial); e
2. exibir um único problema L' já conhecido como NP-completo e mostrar $L' \leq_p L$.

Por que basta um único L' ? Como L' é NP-completo, todo $L'' \in NP$ satisfaz $L'' \leq_p L'$. Se também $L' \leq_p L$, a transitividade (Lema 4.2) dá $L'' \leq_p L$ para todo $L'' \in NP$. Ou seja, um único “elo” basta para herdar as reduções de todos os problemas de NP simultaneamente. $\square$

O Corolário 5.2 é a técnica que vamos aplicar repetidamente na Seção 7. Mas ele tem um problema de “ovo e galinha”: precisamos de *pelo menos um* problema NP-completo estabelecido “do zero” – provando a condição (2) da Definição 5.1 diretamente, contra todos os infinitos problemas de NP de uma vez – para começar a cadeia. Esse é o papel do teorema da próxima seção.

6 O Teorema de Cook-Levin (como caixa-preta)

Teorema 6.1 (Cook–Levin, Cook, 1971). *CIRCUIT-SAT é NP-completo, onde CIRCUIT-SAT(C) pergunta: dado um circuito booleano C com portas AND, OR e NOT, existe uma atribuição das entradas que faz a saída de C ser 1?*

Não vamos demonstrar este teorema linha a linha – a prova completa está em Cormen et al. (2009, Seção 34.3–34.4) para quem quiser se aprofundar. O que importa *para usar* o resultado é a ideia central:

Ideia da prova. CIRCUIT-SAT $\in NP$ é fácil: o certificado é a própria atribuição de entradas, e simular o circuito com essa atribuição é polinomial. A parte difícil é a segunda condição: mostrar que *todo* $L \in NP$ reduz a CIRCUIT-SAT. A observação-chave é que um verificador polinomial $A(x, y)$ é, ele mesmo, um programa que roda em tempo polinomial – e todo programa que roda em tempo T pode ser simulado por um circuito booleano de tamanho polinomial em T , essencialmente “desenrolando” a sequência de configurações da máquina passo a passo em portas lógicas. O circuito resultante recebe x fixado (“cravado” nos fios de entrada correspondentes) e y como entradas livres; ele é satisfazível *exatamente quando* existe y que faz $A(x, y)$ aceitar – ou seja, exatamente quando $x \in L$. Essa construção do circuito a partir do verificador é, ela mesma, computável em tempo polinomial, como a Definição 4.1 exige.

Corolário 6.2. CIRCUIT-SAT é o “problema-semente”: a partir dele, o Corolário 5.2 nos permite provar a NP-completude de qualquer outro problema exibindo apenas uma redução, sem repetir o argumento de simulação de máquinas por circuitos.

7 A receita prática e a cadeia de reduções

Toda prova de NP-completude por redução segue o mesmo roteiro, e vale a pena nomeá-lo antes dos exemplos. Para provar que L é NP-completo:

1. **Mostre $L \in NP$** (geralmente trivial: exiba o certificado e o verificador).
2. **Escolha um L' NP-completo já conhecido**, estruturalmente parecido com L , e monte a redução *na direção certa*: $L' \leq_p L$ (releia a Observação 4.1 se necessário).
3. **Construa f** : uma transformação polinomial de instâncias de L' para instâncias de L , tipicamente por meio de **gadgets** – pedaços da instância de L que “imitam” peças da instância de L' (uma cláusula vira um triplo de vértices; uma variável vira um par de números; etc.).
4. **Prove a correção nas duas direções**: (a) se x é instância “sim” de L' , então $f(x)$ é instância “sim” de L ; e (b) se $f(x)$ é instância “sim” de L , então x é instância “sim” de L' . **Estudantes costumam provar só a direção (a) e esquecer a (b) – ambas são obrigatórias, pois a Definição 4.1 exige um “se, e somente se”.**

Antes das reduções, definimos formalmente os problemas envolvidos.

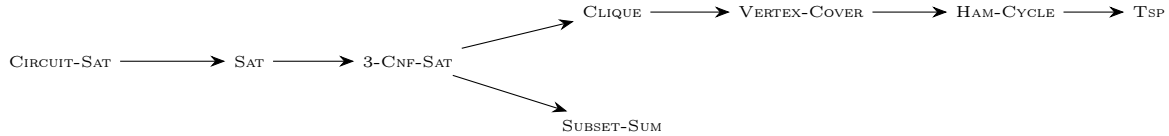
Definição 7.1 (FNC, literal, cláusula). Um **literal** é uma variável booleana x ou sua negação $\bar{x}$ (também escrita $\neg x$). Uma **cláusula** é uma disjunção (OU) de literais, ex. $(x_1 \vee \bar{x}_2 \vee x_3)$. Uma fórmula está em **forma normal conjuntiva (FNC)** se é uma conjunção (E) de cláusulas: $\phi = C_1 \wedge C_2 \wedge \dots \wedge C_k$. A fórmula é **satisfazível** se existe uma atribuição de valores-verdade às variáveis que a torna verdadeira – ou seja, que torna verdadeiro ao menos um literal em *cada* cláusula.

Definição 7.2 (Os problemas da cadeia). • SAT(ϕ): a fórmula booleana ϕ (qualquer) é satisfazível?

- 3-CNF-SAT(ϕ): a fórmula ϕ , em FNC com *exatamente 3 literais por cláusula*, é satisfazível?
- CLIQUE(G, k): G tem clique de tamanho k ?

- VERTEX-COVER(G, k): existe $V' \subseteq V$, $|V'| \leq k$, tal que toda aresta de G tem ao menos uma ponta em V' ? (V' é uma *cobertura de vértices*.)
- HAM-CYCLE(G): G tem ciclo hamiltoniano?
- TSP(G, w, k): dado grafo completo G com pesos w nas arestas e inteiro k , existe uma *turnê* (ciclo hamiltoniano) de custo total $\leq k$? (Versão de decisão do caixeiro-viajante.)
- SUBSET-SUM(S, t): dado um conjunto finito S de inteiros positivos e um alvo t , existe subconjunto $S' \subseteq S$ com $\sum_{s \in S'} s = t$?

A cadeia clássica, montada por Karp (1972) e adaptada de Cormen et al. (2009, Seção 34.5):



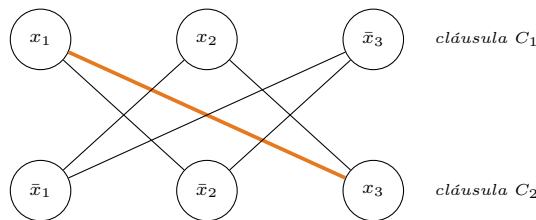
Cada seta $L' \rightarrow L$ é uma redução $L' \leq_p L$; pela transitividade e pelo Corolário 5.2, todos os nós da cadeia são NP-completos (cada um está em NP – verifique mentalmente qual é o certificado de cada um). Vamos detalhar as duas primeiras reduções da cadeia *de grafos* por completo – são as mais instrutivas sobre a técnica de gadgets – e esboçar as demais.

7.1 3-CNF-SAT $\leq_p$ CLIQUE (completa)

Seja $\phi = C_1 \wedge C_2 \wedge \dots \wedge C_k$ uma fórmula em FNC com exatamente 3 literais por cláusula.

Construção do gadget. Para cada cláusula $C_i = (\ell_1^i \vee \ell_2^i \vee \ell_3^i)$, crie um **triplo de vértices** v_1^i, v_2^i, v_3^i , um para cada literal. Ligue v_p^i a v_q^j por uma aresta se, e somente se: (a) estão em triplos diferentes ($i \neq j$); e (b) os literais correspondentes não são complementares (não é o caso de um ser x e o outro $\bar{x}$, para a mesma variável x). A instância de CLIQUE produzida é (G, k) , com $k =$ número de cláusulas de ϕ .

Exemplo 7.1 (Instância concreta). Considere $\phi = (x_1 \vee x_2 \vee \bar{x}_3) \wedge (\bar{x}_1 \vee \bar{x}_2 \vee x_3)$, com $k = 2$ cláusulas. O grafo construído tem 6 vértices e as arestas mostradas abaixo (note as *não*-arestas: pares dentro do mesmo triplo e pares complementares $x_1-\bar{x}_1, x_2-\bar{x}_2, \bar{x}_3-x_3$):



Uma clique de tamanho $k = 2$ é qualquer aresta; a destacada em laranja, $\{x_1, x_3\}$, corresponde à atribuição $x_1 = V$, $x_3 = V$ (e x_2 com valor arbitrário), que de fato satisfaz ϕ : x_1 torna C_1 verdadeira e x_3 torna C_2 verdadeira.

Correção. $(\Rightarrow)$ Se ϕ é satisfazível, fixe uma atribuição satisfatória; cada cláusula tem ao menos um literal verdadeiro sob ela. Escolha um vértice de literal verdadeiro por triplo – são k vértices de triplos distintos. Nenhum par escolhido é de literais complementares (a atribuição não pode tornar x e $\bar{x}$ simultaneamente verdadeiros), então, pela construção, todos os pares escolhidos estão ligados: clique de tamanho k . $(\Leftarrow)$ Se G tem clique de tamanho k : vértices do mesmo triplo nunca são adjacentes, logo a clique tem *exatamente* um vértice por triplo. Atribua valor-verdade de modo a tornar verdadeiro cada literal escolhido pela clique – isso é consistente, pois a clique nunca contém x e $\bar{x}$ simultaneamente (literais complementares não são ligados); variáveis que não aparecem em nenhum literal escolhido recebem valor arbitrário. Essa atribuição torna verdadeiro um literal em cada cláusula, logo satisfaz ϕ .

A construção é claramente polinomial: $3k$ vértices e $O(k^2)$ arestas.

7.2 CLIQUE $\leq_p$ VERTEX-COVER (completa)

Construção. Dada uma instância (G, k) de CLIQUE, com $G = (V, E)$, construa $f(G, k) = (\bar{G}, |V| - k)$, onde $\bar{G}$ é o grafo complementar (Seção 4). Computar $\bar{G}$ custa $O(V^2)$: polinomial.

Correção. Um único fato de teoria dos grafos resolve as duas direções de uma vez:

$$S \subseteq V \text{ é clique em } G \iff V \setminus S \text{ é cobertura de vértices de } \bar{G}.$$

$(\Rightarrow)$ Se S é clique em G , então $\bar{G}$ não tem nenhuma aresta com *ambas* as pontas em S (essas arestas existiriam apenas entre pares não-adjacentes de G). Logo toda aresta de $\bar{G}$ tem ao menos uma ponta fora de S , isto é, em $V \setminus S$ – que é,

portanto, cobertura. ($\Leftarrow$) Se $V \setminus S$ é cobertura de $\bar{G}$, nenhuma aresta de $\bar{G}$ tem ambas as pontas em S ; ou seja, todo par de vértices de S é adjacente em G : S é clique.

Tomando $|S| = k$: G tem clique de tamanho $k \iff \bar{G}$ tem cobertura de tamanho $|V| - k$, exatamente a equivalência que a Definição 4.1 exige.

7.3 As demais reduções da cadeia (esboço)

- **VERTEX-COVER $\leq_p$ HAM-CYCLE**: a mais elaborada da cadeia. Cada aresta de G vira um gadget de 12 vértices (um “trilho” que o ciclo pode atravessar de três maneiras, codificando qual das pontas da aresta está na cobertura) e adicionam-se k vértices “seletores” que forçam a escolha de exatamente k vértices para a cobertura. Ver Cormen et al. (2009, Seção 34.5.3) para o gadget completo.
- **HAM-CYCLE $\leq_p$ TSP**: quase imediata – e ótima para fixar a receita. Dado G com n vértices, construa o grafo completo K_n com pesos $w(u, v) = 0$ se $(u, v) \in E(G)$ e $w(u, v) = 1$ caso contrário; a instância de TSP é $(K_n, w, 0)$. Uma turnê de custo 0 só pode usar arestas de peso 0, ou seja, arestas de G – e uma turnê que só usa arestas de G é exatamente um ciclo hamiltoniano de G . **Exercício mental: escreva as direções (a) e (b) da correção explicitamente para esta redução.**
- **3-CNF-SAT $\leq_p$ SUBSET-SUM**: cada variável e cada cláusula viram números decimais (um dígito posicional por variável e por cláusula) construídos de forma que a soma-alvo t só é atingível escolhendo números que correspondem a uma atribuição satisfatória. Fica como leitura complementar (Cormen et al., 2009, Seção 34.5.5). Esta redução ilustra bem por que a *codificação em binário* da entrada (Seção 2) importa: SUBSET-SUM admite programação dinâmica em $O(n \cdot t)$ – **pseudo-polinomial**, pois t pode ser exponencial no número de *bits* da entrada. Se os números fossem dados em unário, esse algoritmo seria polinomial e o problema não seria NP-completo (assumindo $P \neq NP$); a intratabilidade depende criticamente da representação compacta.

8 E agora? Vivendo com problemas NP-completos

Reconhecer que um problema é NP-completo não é um beco sem saída – é uma mudança de estratégia. Na prática, um engenheiro ou pesquisador diante de um problema NP-completo tem várias saídas legítimas:

1. **Algoritmos exponenciais, mas rápidos na prática**: *branch-and-bound*, *backtracking* com poda agressiva, programação por restrições. O pior caso continua exponencial, mas instâncias reais raramente atingem o pior caso – é assim que *SAT solvers* modernos resolvem, rotineiramente, instâncias industriais com milhões de variáveis.
2. **Casos especiais tratáveis**: muitos problemas NP-completos no caso geral tornam-se polinomiais quando a entrada tem estrutura restrita (grafos bipartidos, planares, de largura em árvore limitada). Vale sempre perguntar: “minha instância real tem alguma estrutura especial?”
3. **Algoritmos de aproximação**: abrir mão da otimalidade em troca de garantia formal de proximidade. Ex.: existe algoritmo 2-aproximado para VERTEX-COVER (tome as duas pontas de cada aresta de um emparelhamento maximal) – garantidamente polinomial, garantidamente no máximo $2 \times$ o ótimo.
4. **Heurísticas e metaheurísticas**: busca local, *simulated annealing*, algoritmos genéticos – sem garantia formal, mas frequentemente excelentes na prática.
5. **Complexidade parametrizada**: se o problema tem um parâmetro k pequeno (não necessariamente o tamanho da entrada), pode existir algoritmo $O(f(k) \cdot n^c)$ – eficiente quando k é pequeno, mesmo que a entrada n seja grande.

A mensagem final: NP-completude não é uma sentença de “impossível”, é uma sentença de “não espere garantia polinomial de pior caso – escolha conscientemente qual concessão você vai fazer (tempo, otimalidade ou generalidade)”.

Referências

- Cook, S. A. (1971). The complexity of theorem-proving procedures. In *Proceedings of the Third Annual ACM Symposium on Theory of Computing (STOC)*, pages 151–158.
- Cormen, T. H., Leiserson, C. E., Rivest, R. L., and Stein, C. (2009). *Introduction to Algorithms*. MIT Press, Cambridge, MA, 3 edition.
- Garey, M. R. and Johnson, D. S. (1979). *Computers and Intractability: A Guide to the Theory of NP-Completeness*. W. H. Freeman, New York.
- Karp, R. M. (1972). Reducibility among combinatorial problems. In Miller, R. E. and Thatcher, J. W., editors, *Complexity of Computer Computations*, pages 85–103. Plenum Press.